



USC University of
Southern California

LLMs as Agents

The Model Layer

CSCI 599 — Agentic AI · Lecture 3 / Week 2

Prof. Marco Papa



Last lecture, GPT-5.5 did the work

The Codex desktop app you used in the vibe-coding demo

- Underneath: GPT-5.6 Sol — now USC's Codex default
- The model decides which tool to call. Codex routes the call.

You saw the methodology. Today: the layer underneath.

Three questions for today

What makes a model an agent backend, not just a chatbot?

Which model fits which node in the graph?

What does it look like to swap one for another?

LangGraph: The Graph



Why graphs for agents?

Agents loop. Pipelines don't.

- A user query becomes: model → tool → model → tool → answer
- Each step depends on the last step's output
- Errors can trigger retries when a retry policy is configured

State-graph model fits the shape. DAGs don't.

StateGraph: three primitives

State: a schema shared by every node

- Nodes receive current state and return partial state updates; reducers determine how updates are combined

Nodes: Python functions that read state, return updates

- Side effects are possible, but retry and replay semantics require explicit design

Edges: how control flows between nodes

- Fixed edge — always go to node X next
- Conditional edge — routing function inspects state and returns the next node

The smallest LangGraph

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_google_genai import ChatGoogleGenerativeAI

class State(TypedDict):
    question: str
    answer: str

def call_model(state: State) -> dict:
    llm = ChatGoogleGenerativeAI(model="gemini-3.5-flash")
    reply = llm.invoke(state["question"])
    return {"answer": reply.content}

graph = StateGraph(State)
graph.add_node("model", call_model)
graph.add_edge(START, "model")
graph.add_edge("model", END)
app = graph.compile()
```

THE SHAPE

One node. One model. End.
Everything else is elaboration on this shape.

The seam: the model is one node

ChatGoogleGenerativeAI in the example — easily swapped

- ChatAnthropic for Claude. ChatOpenAI for GPT.
- ChatOllama for local — Gemma 4, Llama, Mistral, anything Ollama serves.

Same graph, different backend. State schema unchanged.

Today's demo lives at this seam.

PAPA'S RULE

Pick the model that fits the node — not the brand that fits your slack channel.

Tool calls live at the model node

```
tools = [search_web, query_db, send_email]
llm = ChatAnthropic(
    model="claude-sonnet-5"
).bind_tools(tools)

def call_model(state):
    response = llm.invoke(state["messages"])
    # response.tool_calls -> list of {name, args}
    return {"messages": [response]}
```

The model decides

It emits structured tool_calls in its response — a list of {name, args}.

The graph routes

Your conditional edge inspects tool_calls and sends the call to your Python function.

Which model matters

Tool-call fidelity is part of agent quality — not raw benchmark IQ.

The Model Layer



What makes a good agent backend?

Primary — what makes a backend useful for agentic work

Tool-call fidelity

- Does the model reliably emit syntactically valid tool names and arguments?

Structured action reliability

- Does the model consistently follow the tool schema and return usable arguments?

MCP interoperability

- How easily can the surrounding agent stack discover and invoke MCP tools?

Secondary — the trade-offs once the primary axes are met

- Latency under loops · Context window · Cost per step

Cloud models — landscape today

Model	Released	Context	\$/1M in / out	Trait
Gemini 3.6 Flash	Jul 2026	1M	\$1.50 / \$7.50	Efficient production agent work
GPT-5.6 Sol	Jul 2026	1.05M	\$5 / \$30	OpenAI flagship
Claude Opus 5	July 2026	1M	\$5 / \$25	Long-running coding and agents
Claude Sonnet 5	Jun 2026	1M	\$2 / \$10*	Strong cost/capability balance
Grok 4.5	Jul 2026	500K	\$2 / \$6	Coding and agentic workflows
Gemini 3.5 Flash	May 2026	1M	\$1.50 / \$9	Course demo baseline

Snapshot: Aug. 13, 2026. *Sonnet 5 introductory pricing runs through Aug. 31, 2026.

Gemini 3.5 → 3.6 Flash — the update pattern

Google's Flash family — course demo uses 3.5 Flash for cost

- 3.5 Flash: frontier performance for agents with fast action
- 3.6 Flash: 17% fewer output tokens, with lower output-token cost
- Efficiency gains matter when an agent loops and calls tools repeatedly

3.5 Pro is currently in testing; Gemini 4 pre-training has begun

Gemini 3.7 Flash released August 13, 2026

Gemini 3.6 Flash — measured improvements

Benchmark	3.5 Flash	3.6 Flash	What it measures
DeepSWE	37%	49%	Long-horizon software engineering
MLE Bench	49.7%	63.9%	Machine learning engineering
OSWorld-Verified	78.4%	83.0%	Agentic computer use
GDPval-AA v2	1349	1421	Knowledge-work benchmark

3.6 Flash improves on every benchmark shown here, while using 17% fewer output tokens. Same Flash tier, more capability per token.

Gemini 3.5 Flash — in code

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-3.5-flash",
)

# Bind tools the model can call
llm = llm.bind_tools([search_web, query_db])

response = llm.invoke([
    {"role": "user", "content": "Find Marco's office hours."}
])

# response.tool_calls -> [{"name": "search_web", "args": {...}]}
```

Claude — Sonnet first, Opus when you can name why

Sonnet 5 (intro through Aug 31): \$2 / \$10 per 1M · Opus 5: \$5 / \$25

Opus 5 is 2.5× Sonnet 5's current introductory token price

Opus 5: 1M context, hybrid reasoning, strong agentic performance

Sonnet 5 can match Opus 5 on some higher-effort tasks at lower cost

PAPA'S RULE

Pick Sonnet 5 first. Reach for Opus 4.8 only when you can name the specific reasoning or reliability the agent needs that Sonnet missed.

GPT-5.6 Sol — now USC's Codex default

OpenAI flagship launched July 9

- Sol (\$5/\$30), Terra (\$2.50/\$15), Luna (\$1/\$6) per 1M tokens

USC Codex now defaults to GPT-5.6 Sol

- Sol is the frontier tier for complex professional and agentic work
- Context window: 1.05M tokens; requests above 272K input tokens use higher rates

Strong tool integrations across the OpenAI platform

Local Models



Why local matters for agents

Privacy — with fully local inference, data can remain on-device

- Useful for sensitive data when policy permits on-device inference

Offline — agent loops can continue without a cloud connection

- Edge deployments, mobile, intermittent connectivity

No per-token API bill for local inference

- Hardware, power, and maintenance still have real costs

Gemma 4 family

Variant	Parameters	Context	Use case
gemma4:e2b	2.3B effective	128K	Edge devices, mobile
gemma4:e4b	4.5B effective	128K	Laptops — today's demo backend
gemma4:12b	12B dense	256K	Prosumer machines
gemma4:26b	25.2B total / 3.8B active MoE	256K	Workstations, hybrid front-end
gemma4:31b	30.7B dense	256K	Workstations, capable local alternative

E = effective parameters. MLX variants are available for Apple Silicon, including gemma4:e4b-mlx, gemma4:26b-mlx, and gemma4:31b-mlx.

Modalities vary by size: E2B/E4B support text, image, and audio; the larger 26B/31B variants support text and image.

Gemma 4 via Ollama

```
# Install once
$ ollama pull gemma4:e4b

# In Python – same LangChain interface as cloud models
from langchain_ollama import ChatOllama

llm = ChatOllama(
    model="gemma4:e4b",
)

# Same bind_tools interface as cloud models
llm = llm.bind_tools([search_local_db])

response = llm.invoke("Find Marco's office hours in the syllabus.")
```

Preview: the `|think|` token

Gemma 4 has a configurable thinking mode. One control token enables it.

```
messages = [  
    {"role": "system", "content": "<|think|>"},    # thinking ON  
    {"role": "user", "content": "Solve: ..."}  
]  
# Model emits a thought block, then the final answer.
```

Next lecture: prompting techniques. The `|think|` token is one of five we'll run against the same task — zero-shot, few-shot, chain-of-thought, configurable thinking, ReAct.

Wiring It Up



The seam, recap

One graph, one node, one model

Swap the model — the graph doesn't notice

Cloud vs local, frontier vs edge — same wire shape

Next slide is the code. Then we run it.



Demo: one graph, three backends

```
def build_llm(backend: str):
    if backend == "gemini":
        llm = ChatGoogleGenerativeAI(
            model="gemini-3.5-flash")
    elif backend == "gemma4":
        llm = ChatOllama(model="gemma4:e4b")
    elif backend == "gemma4-26b":
        llm = ChatOllama(model="gemma4:26b")

    return llm.bind_tools([search_arxiv])

g = StateGraph(AgentState)
g.add_node("model", make_model_node(llm))
g.add_node("tool", tool_node)
g.add_conditional_edges("model", needs_tool)
g.add_edge("tool", "model")
```

WHAT TO WATCH

Same graph. The --backend flag changes the model. Watch latency and tool-call shape.

What the swap showed

Recorded baseline runs from the executable demo:

- Gemini 3.5 Flash: 16.31s, 4 sequential tool calls — refined the query each turn
- Gemma 4 e4b: 11.47s, 1 tool call — explicitly set max_results=2
- Gemma 4 26B: 24.34s, 2 tool calls issued in one model response

Different planning strategies — compare the shape, not just the clock.

Hybrid routing: the architecture

Front-end: gemma4:e4b on the device — cheap, fast, private

- Handles the common case locally when the workload permits

Escalation: Gemini 3.6 Flash or Claude Sonnet 5

- Triggered by confidence drop, complexity threshold, or tool failure

One LangGraph, two model nodes, a conditional edge between them

- Same architectural pattern as the demo. One step bigger.

This is the spine of Assignment 3

Today: prove the seam exists. Three backends, one graph.

Assignment 3: build the routing logic.

- Local Gemma 4 front-end + cloud escalation
- Confidence-based routing decision
- Persistent state across turns

Start with the demo code. Build the routing layer when the assignment releases.

Model selection cheat sheet

Node type	First pick	When to escalate
High-volume routing / classification	gemma4:e4b local	Confidence drops on your validation set
Tool-heavy reasoning step	Gemini 3.7 Flash	Tool failures or reasoning quality miss your target
Long-context synthesis	Claude Sonnet 5	Measured workload needs greater reasoning depth
Multi-step plan with branching	Claude Opus 5 / GPT-5.6 Sol	Capability gap is demonstrated on the task

PAPA'S RULE

Hybrid is the answer. Local for cheap, fast, private. Cloud for capability you can name. Validate the routing policy on your workload.

Next: the lever, not the substrate

Today was the substrate — which model lives in the node

Next lecture: the lever — how to make any model behave differently

- Zero-shot → few-shot → chain-of-thought
- Configurable thinking — Gemma 4's |think| token in action
- ReAct — preview of the agent loop

Same task, five techniques, one model. Watch what changes.